A Systematic Evaluation of Environmental Flakiness in JavaScript Tests

Negar Hashemi*, Amjed Tahir*, August Shi†, Shawn Rasheed‡, Rachel Blagojevic*
*Massey University, New Zealand
‡Victoria University of Wellington, New Zealand
† The University of Texas at Austin, USA

arXiv:2602.19098v1 [cs.SE] 22 Feb 2026

***Abstract*—Test flakiness is a significant issue in industry, affecting test efficiency and product quality. While extensive research has examined the impact of flaky tests, many root causes remain unexplored, particularly in the context of dynamic languages such as JavaScript. In this paper, we conduct a systematic evaluation of the impact of environmental factors on test flakiness in JavaScript. We first executed test suites across multiple environmental configurations to determine whether changes in the environment could lead to flaky behavior. We selected three environmental factors to manipulate: the operating system, the Node.js version, and the browser. We identified a total of 65 environmental flaky projects, with 28 related to operating system issues, five to Node.js version compatibility, 16 to a combination of operating system and Node.js issues, and 17 related to browser compatibility. To address environmental flakiness, we developed a lightweight mitigation approach, `js-env-sanitizer`, that can *sanitize* environmental-related flaky tests by skipping and reporting them (rather than failing), allowing CI builds to continue/succeed without rerunning entire test suites. The tool achieves high accuracy with minimal performance or configuration overhead, and currently supports three popular JavaScript testing frameworks (Jest, Mocha, and Vitest).**

I. Introduction

Flaky tests are tests whose outcomes alternate between pass and fail without any changes to the code. They are recognized as a phenomenon that continuously causes significant problems in industry [1], [2] and negatively impacts product quality and delivery [3], [4], as well as testing-related activities [5], [6]. Several reports from tech companies highlighted the high cost and impact of flaky tests on their projects, including reports from Google [7], Microsoft [8], and GitHub [9].

Environmental flakiness refers to test failures that are caused not by issues in the test logic or application code itself, but by variations in the environment in which the tests are executed. Environmental factors may include any variations in the operating systems (OS), hardware configurations, available system resources, or browsers [10]. When a test passes in one environment but fails in another without any code changes, it is often considered to be environmentally flaky. This type of flakiness is particularly challenging to detect, as it may depend on subtle, non-deterministic interactions with the system. For example, a test might fail intermittently on macOS but pass consistently on Linux, or it might behave differently when executed in a container than in a native environment.

As modern software systems increasingly rely on distributed, containerized, and cross-platform deployments, managing environmental variability becomes critical to ensuring stable and trustworthy test results. Test consistency across environments is important as any flakiness resulting from environmental variation can undermine the reliability of continuous integration (CI) pipelines, delay deployments, and introduce unnecessary engineering effort [11].

Previous studies have identified and categorized potential causes of flaky tests across different programming languages, including Java [12], Python [13], and JavaScript [14]. Hashemi et al. [14] reported the first extensive empirical study on flaky tests in open-source JavaScript projects, categorizing the causes and fixing strategies for these flaky tests. They found that one of the major causes of flakiness in JavaScript is the environment in which the tests are run (a similar observation was reported in Costa et al. [15]).

In this study, we conducted an in-depth investigation into environmental flakiness in JavaScript, focusing on the OS, browsers, and Node.js versions. Our goal is to understand the prevalence of flakiness by changing the environmental factors. We also want to understand the reasons for the failure after running tests in various environment configurations. Based on our empirical results, we aim to develop a technique that can help developers mitigate environmental flakiness. In practice, developers are reported to employ several methods to address test flakiness, such as adding suitable assumptions to the tests to run them in specific environments or with specific configurations [14], [10]. Developing an automated technique to sanitize tests against environmental conditions can help developers gain more control over the tests' behavior.

The main contributions of this paper are the answers to the following research questions:

RQ1 How prevalent are environmental flaky tests in JavaScript?

RQ2 What are the root causes of environmental flakiness?

RQ3 How can we mitigate test flakiness caused by environmental factors?

RQ1 aims to investigate the prevalence of environmental flakiness in JavaScript. We employed a systematic rerun approach to uncover environmental flaky tests. By changing environmental conditions and rerunning the tests under different conditions, we assess whether the test outcomes change. RQ2 aims to understand the main causes of environmental flaky tests in JavaScript. We manually inspected each detected environmental flaky test and classified the cause of flakiness into different categories. RQ3 aims to identify a mitigation

method for flaky tests caused by environmental variation.

Based on our systematic evaluation of possible environmental flakiness in the 116 projects we investigated, we identified a total of 39 environmental flaky projects (i.e., flakiness that occurs at the project level due to specific project setups or package dependencies). We also identified 28 environmental flaky tests (i.e., tests that fail due to the change in the test environment (OS, Node.js version and/or the browser)). Looking more closely at the causes, we found that 28 flaky tests are related to operating system issues, five are related to Node.js version dependencies, and 17 are related to browser setup. To mitigate such environmental flakiness causes by variation of OS, Node.js, or browsers, we developed an annotation-based approach, implemented in a tool called `js-env-sanitizer`, that can sanitize and quarantine such flaky tests by skipping and reporting them (instead of failing the tests), which allows CI builds to continue/succeed without the need to rerun entire test suites or stopping the build for developers to inspect (flaky) failures. Our tool is framework-independent and works as a plugin that currently supports three popular JavaScript testing frameworks (Jest [16], Mocha [17], and Vitest [18]). We uncovered that `js-env-sanitizer` helps skip and report all tests that typically fail due to environmental variation.

II. Background

The environment in which programs are executed can significantly influence their behavior, which, in turn, can affect test execution and lead to changes in test outcomes depending on the environment configuration. These tests are therefore *flaky* because their outcomes depend not only on the code they test but on external environmental factors. We refer to such tests as *environmental flaky tests*: tests that fail in one environment but pass in others (with other variables in control, including code). These failures may be consistently tied to a specific environment or occur intermittently. We consider compatibility issues as a subset of environmental flakiness, as they manifest as failures that occur only in certain environments (e.g., due to differences in API implementations across JVMs, while the calling code remains unchanged [19]).

A. Environmental Factors

In the context of our study, we considered three environmental factors that may contribute to test flakiness in JavaScript: (1) the OS, (2) the version of the runtime environment (Node.js), and (3) the browsers.

1) Operating Systems: OSs are a major cause of flakiness in JavaScript [14], with Ubuntu, macOS, and Windows being the most commonly used platforms [20]. Differences between OS can lead to inconsistent test behavior due to variations in properties such as file systems (e.g., path separators, case sensitivity), line endings, default locale and encoding, performance and timing, and permission models [10], [21].

2) Node.js: Node.js [1] is a cross-platform JavaScript runtime environment popular in the JavaScript community [22]. Node.js allows developers to execute JavaScript code outside of a web browser. It is widely adopted in open-source and enterprise projects, with a rich ecosystem of packages supported through the Node Package Manager (npm). Node.js follows semantic versioning (SemVer), where major releases can introduce breaking changes. With the numerous available Node.js versions, it is important to consider how these different versions may impact test outcomes. The JavaScript engine and built-in APIs may change and update between Node.js versions. These changes can affect how code executes, especially in terms of timing and asynchronous behavior, which may expose hidden race conditions or timing-related issues in tests.

3) Browsers: Browsers introduce another source of environmental variability for tests. A large number of JavaScript projects involve processing and rendering browser-related events. Different browsers make slightly different choices about rendering, timing, caching, and permissions. These differences can cause a test to pass in one browser but intermittently fail in another. In addition, the interaction between OS and browser can further contribute to test flakiness, as they may behave differently depending on the underlying OS. For example, a browser feature might work reliably on macOS but behave inconsistently on Windows due to differences in OS-level APIs or hardware acceleration support.

B. GitHub Actions

We utilised GitHub Actions to run our experiment, enabling us to systematically explore the impact of environmental variations on test outcomes. GitHub Action[2] is a powerful automation tool integrated directly into GitHub that enables developers to define and execute custom workflows for CI [23].With a GitHub Actions workflow, developers can automate tasks such as building, testing, and deploying code whenever changes are pushed to a repository. GitHub Actions supports a variety of OS, including Ubuntu, macOS, and Windows, and it provides access to pre-installed environments with popular development tools and languages. For our experiments, we set up a GitHub Actions workflow to run tests from each project across multiple OS (Ubuntu, macOS, and Windows), Node.js versions, and browsers in consistent, isolated environments.

III. Methodology

We conducted our experiment in two phases: (1) we ran tests from each project under different environment configurations to see whether tests have different outcomes, and then (2) we manually analyzed the flaky failures and categorized the causes of flakiness. Figure 1 shows an overview of our experiment, including the data collection and analysis process.

A. Dataset

We constructed a dataset of JavaScript projects by using GitHub's REST API [24] to collect the most popular projects ranked by GitHub stars. GitHub stars are a metric that indicates the level of interest a project has received, with prior work finding a correlation between the number

[1]https://nodejs.org/en

[2]https://docs.github.com/en/actions

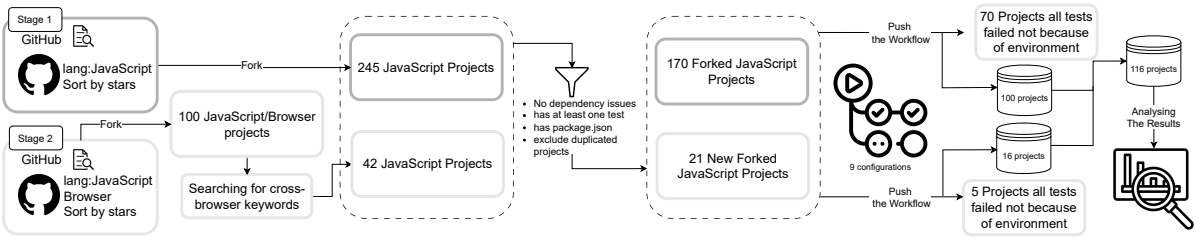


Fig. 1: An overview of the data collection and analysis process

of stars and the number of forks and contributors [25]. We selected the 245 most starred JavaScript projects to form a large enough initial dataset for inspection. We then filtered out projects that do not have any tests or do not utilize the Node.js runtime environment, which resulted in 170 JavaScript projects. Note that these projects may use different JavaScript testing frameworks (e.g., Jest, Mocha, and Vitest).

After we forked the 170 projects, we committed and pushed a GitHub Actions workflow into our forked repository to allow us to run these tests ourselves in different environments using the GitHub Actions CI. We noted that some runs can fail because of dependency errors or blocked running actions in the projects. We manually customized the workflow to fix dependency errors or failed tests. For example, some projects could not be installed without using the `--force` flag, while others required a specific library/tool version (e.g., Python 2) to be installed that was not available by default. We excluded any projects where we could not build or manually fixed, leaving us with 100 projects in our dataset.

Since one of the primary applications of JavaScript is for scripting in web browsers, we aim to investigate how the combination of OS and browsers contributes to test flakiness. However, we find that our dataset contained only 22 web-based JavaScript projects. Therefore, we conducted an additional search on GitHub to identify and locate projects specifically designed to run cross-browser tests. By filtering for projects containing the keyword “browser”, we selected the 100 most-starred repositories to expand our dataset. To determine whether a project includes cross-browser testing, we examined each project’s package file for browser-specific keywords. We selected a set of terms commonly associated with tools and frameworks used for browser-based or end-to-end testing, including: “playwright”, “karma-chrome-launcher”, “karma-firefox-launcher”, “karma-safari-launcher”, “selenium-webdriver”, “webdriverio”, “testcafe”, “cypress”, “test:e2e”, “gulp” and“grunt”. This search identified 42 projects, 11 of which were already available in our initial dataset. We apply the same processing steps used for the initial dataset to the remaining 31 projects. After excluding overlapping projects between the two datasets, we construct an additional dataset consisting of 16 new projects. Overall, our combined dataset contains 116 projects. Table I summarizes the statistics of the projects in our dataset.

TABLE I: Project Statistics

Stats	#Stars	#Forks	#Contributors	#Test Files
Mean	35411.27	5482.19	314.78	63.72
Total	4107707	635934	36515	7392

B. Procedure

With our dataset of JavaScript projects, we set up a procedure that can run tests from those projects to check whether variance in environmental factors can change test outcomes. For each project, we created a GitHub Actions workflow file to run tests in different environments. In this file, we define different combinations of environmental factors (Listing 1). This file enables the GitHub CI pipeline to run each project across nine distinct environment configurations, with each configuration rerun 10 times, resulting in a total of 90 runs.

```
strategy:
    fail-fast: false
    matrix:
      os: [ubuntu-latest, macos-latest, windows-latest] # Operating systems to test
      node-version: [18, 20, 22] # Node.js versions to test
      attempt: [1, 2, 3, 4, 5, 6, 7, 8, 9,10] # Repeat tests 10 times
```

Listing 1: Different environmental setups in GitHub Actions

Table II shows the list of the selected OS, Node.js versions, and browsers available for GitHub Actions[3]. For OS, we use the latest versions available on GitHub Actions. For Node.js, we select the latest stable releases (versions 18, 20, and 22). Ultimately, we pair the different versions of OS and Node.js to form configurations. The three primary OSs (Ubuntu 24.04, macOS 14, and Windows Server 2022) each include multiple pre-installed browsers, such as Google Chrome, Microsoft Edge, Mozilla Firefox, and Safari (exclusive to macOS). Browser versions vary slightly across these OS, affecting feature support and rendering behavior. For instance, Safari is available only on macOS, whereas Microsoft Edge and Firefox are available on both Ubuntu and Windows. We therefore cannot explicitly change browser versions, as they are tightly coupled with the OS.

We manually customize the GitHub Actions workflow for each project to ensure compatibility and successful execution.

[3]https://github.com/actions/runner-images\#available-images

TABLE II: Default OS, Node.js, and Browser in GitHub Actions runners (at the time of running the experiment)

Label	OS	Node.js	Default Browser
ubuntu-latest	Ubuntu 24.04	18.20.8 20.19.1 22.15.0	Google Chrome 136.0.7103.92 Microsoft Edge 136.0.3240.64 Mozilla Firefox 138.0.1
macos-latest	macOS 14	18.20.8 20.19.1 22.15.0	Google Chrome 136.0.7103.49 Safari 18.4 (19621.1.15.111.1)
windows-latest	Windows Server 2022	18.20.8 20.19.1 22.15.0	Google Chrome 136.0.7103.93 Microsoft Edge 136.0.3240.64 Mozilla Firefox 138.0.1

These adjustments help ensure that the test environment accurately reflects the project's expected setup and minimizes configuration-related failures that are unrelated to the environmental factors we vary. The process includes setting the correct branch name when the default is not `main`, as some projects use alternative names such as `master` or custom branches for active development. Additionally, we adapted the installation steps for each project to meet its specific requirements. For instance, some projects require running custom scripts, using different package managers (e.g., `pnpm` vs. `npm`), or installing additional dependencies, such as system packages or global tools, before tests can run.

After running each project across all environmental configurations, we saved the results of each rerun under the name of the corresponding forked project. For some failing runs due to dependency errors or blocked GitHub Actions runs, we manually customize the workflow to fix dependency errors or failed tests. For example, Bootstraprequires JDK (Java) 17, but the default JDK in GitHub Actions on Windows Server 2022 is Java 8. Therefore, we manually modify the workflow to install JDK 17. After customizing the GitHub Actions workflow for each project, we commit the changes to our forked repository. Pushing these changes triggers the GitHub Actions pipeline, which automatically starts running the test jobs.

C. Manual Analysis

We manually reviewed the results obtained from all 116 projects. For every observed test failure in GitHub Actions, two of the co-authors independently examined the error message and traced the relevant code to determine the root cause. If a failure was attributable to the OS, Node.js version, browser, or an interaction between them, we categorized it as environmental flakiness and assigned it to the corresponding category. In cases where the two authors disagreed, a third author was involved to resolve the conflicts. All disagreements were resolved through discussion until full consensus was reached. In total, the two authors were able to reach consensus for 56 projects, and the third author needed to be involved for only 12 projects. In total, we observed test failures from 66 projects in which 65 are environmental flaky projects and five are flaky, caused by other factors, with four projects overlapping between the two categories.

IV. Results

We present the results in relation to the first two RQs below.

A. Prevalence of Environmental Flakiness in JavaScript (RQ1)

By analyzing 116 projects, we categorise those projects into four groups based on the flaky tests we detected in those projects across different environmental configurations (i.e., OS type and Node.js version):

- **Non-flaky projects**: projects that exhibit no test failures when running across different environmental configurations.
- **Environment-dependent projects**: projects with test failures caused by changes in the environment. These are further divided into:
  - **Environmental flaky projects**: failures occur due to environment configurations at the project level, e.g., the project's package dependencies, before tests are even run.
  - **Environmental flaky tests**: only specific tests fail under certain environment configurations, while the rest of the project remains stable.
- **Flaky projects**: projects that contain non-environment-related flaky tests beyond the factors that we control for, e.g., failures caused by network instability or randomness.

In total, from all 116 projects, we found 65 environmental dependent projects, including 39 environmental flaky projects and 28 projects with environmental flaky tests. We detect 1355 environment-dependent tests within those 28 projects. On the other hand, 50 projects are non-environment-dependent. Most environmental flaky failures manifested on the first rerun after a change in the environment (77% of projects). We required 10 reruns in the worst-case scenario.

Table III indicates the number of projects in different categories, as well as the number of causes in each. Note that a single project may belong to more than one group if it exhibits multiple types of behavior. For example, a project may have tests that fail only in specific environments (categorized as environmental flaky tests) and also include tests that fail intermittently, regardless of the environment (categorized as flaky projects). As such, the summation of projects across all categories exceeds the total number of projects.

Environmental flaky projects include different causes of flakiness: 21 are OS dependent, three are Node.js version dependent, and eight are browser dependent. In addition, the causes of test-level flakiness include seven OS dependent, two Node.js-dependent, and nine browser-dependent flaky tests.

In addition, five projects are flaky due to non-environment-related causes beyond those that we control for. For example, Listing 2 shows an asynchronous setup function that intermittently fails with the `error: Timeout - Async function did not complete within 10000ms (set by jasmine.DEFAULT_TIMEOUT_INTERVAL).` This failure occurs when the `await` operation, such as clearing the database, takes longer than Jasmine's default 10-second timeout, commonly due to slow database responses.

```
1 beforeEach(async () => {
```

TABLE III: Projects with/without environmental dependencies

	#projects	#NoD*	#OSD**	#BrD***	#NoD & OSD
Non-flaky projects	50	-	-	-	-
Environmental flaky projects	39	3	21	8	8
Environmental flaky tests	28	2	7	9	8
Flaky projects	5	-	-	-	-
Total	122	5	28	17	16

* Node dependent
** OS dependent
*** Browser dependent

```
2     await new MongoStorageAdapter({ uri: ↩
          databaseURI }).deleteAllClasses();
3     Config.get(Parse.applicationId).schemaCache↩
          .clear();});
```

Listing 2: Failure caused by async wait

B. Root Causes of Environmental Flakiness (RQ2)

By analyzing the failures observed across the 65 environmental dependent projects, we investigate the root cause of the test failures and possible causes of test flakiness. We categorized the detected flaky tests into four groups: (1) OS dependency, (2) Node.js version dependency, (3) browser dependency, and (4) combination of the above factors.

We next discuss the results concerning the four categories.

1) OS dependency: Tests in this category fail when run on a specific OS due to differences in file system behavior (e.g., case sensitivity in file paths), line endings, shell command compatibility, or platform-specific dependencies. In total, we identify 28 environmental dependent projects with failures caused by running on a different OS. Of these projects, 21 exhibit flakiness at the project level, while seven are environmental flaky tests. Regarding failures on which OS, most failures occur on Windows (22 out of 26 projects), followed by macOS (8) and Ubuntu (2). Some projects exhibit failures on more than one OS.

```
1  "test": "npm run --silent test:config && ↩
       ↪ npm run --silent test:config:base",
2  "test:config": "cd ↩
       ↪ packages/eslint-config-airbnb; npm ↩
       ↪ test",
3  "test:config:base": "cd ↩
       ↪ packages/eslint-config-airbnb-base; ↩
       ↪ npm test",
```

Listing 3: Package.json using Unix-style cd commands

An example of *airbnb/javascript* with OS dependency is shown in Listing 3. The listing shows part of the `package.json` file from the Airbnb JavaScript Style Guide projectthat fails on Windows. The build fails because the `test:config` and `test:config:base` scripts (lines 2 and 3) use Unix-style `cd` commands with semicolons (`;`) to chain commands. On Unix-based systems, the semicolon acts as a command separator, but Windows does not recognize it in the same way. As a result, the shell misinterprets the command and fails to execute the second part of the command. This difference in interpretation leads to errors such as *"The system cannot find the path specified"* and causes the test script to fail. A cross-platform fix would involve replacing the semicolons with `&&`, which works consistently on both Unix-based systems and Windows.

```
1  it('should output the correct snippet ids', ()↩
       => {
2    expect(snippetData.map(({ id }) => id).sort()↩
         ).toEqual(
3      [
4        'articles/s/web-development-tips',
5        'js/s/array-grouping',
6        'js/s/array-initialize',
7        'js/s/array-map-foreach',
8        'js/s/array-compare',
9        'css/s/content-centering',
10       'css/s/css-reset',
11     ].sort());});
```

Listing 4: OS-dependent test using file paths with backslashes

Another example of path-specific issues is shown in Listing 4 (30-seconds-of-code).The test fails on Windows because the OS returns file paths using backslashes (`\`) instead of forward slashes (`/`). This mismatch causes the test to fail when comparing the actual output to the expected snippet IDs. To avoid such issues, the test code should normalize file paths using utilities such as `path.posix` or `path.normalize()` to ensure consistent comparisons across platforms.

```
1  "scripts": {
2     "test": "TZ=Pacific/Auckland npm run ↩
          ↪ test-tz && TZ=Europe/London npm ↩
          ↪ run test-tz && ↩
          ↪ TZ=America/Whitehorse npm run ↩
          ↪ test-tz && npm run test-tz && ↩
          ↪ jest",
```

Listing 5: Package.json using Unix-style cd commands

A similar issue occurs in the *dayjs* project (Listing 5), where the script fails on Windows due to platform-specific handling of environment variables. The script sets the `TZ` (timezone) environment factor inline before calling `npm run test-tz` multiple times. This syntax (`TZ=... command`) is available on Unix-based systems, where one can set environment variables inline. However, this syntax is not natively supported on Windows, resulting in the test failing. To fix this, tools like `cross-env` [26] can be used to set environment factors in a way that works across different OS.

2) Node.js Version Dependency: We identified five test failures that occur across different Node.js versions (i.e., a test passes in one Node.js version but fails in another). Those failures are typically caused by breaking changes in the API, deprecated APIs, stricter error handling, or differences in built-in module implementations. For example, a test may rely on undocumented behavior that changes between Node.js versions. In total, we identified four environmental dependent projects with failures resulting from differences in Node.js versions. Two of these projects are environmental flaky projects,

and two projects include environmental flaky tests, where only specific tests fail depending on the Node.js version.

Listing 6 shows a test from the `pug` repository that expects the error message to follow a specific structure. The test passes when run with Node.js version 15 but fails when run with versions 18, 20, and 22 due to changes in the way error messages are generated [27]. When running the test on version 18, the test shows an error message "Cannot read property 'length' of undefined", as it is expecting a `null` value.

```
it('includes detail of where the error was thrown including the filename', function() {
  var err = getFileError(
    __dirname + '/fixtures/runtime.with.mixin.error.pug',{});
  expect(err.message).toMatch(/mixin.error.pug:2/);
  expect(err.message).toMatch(/Cannot read property 'length' of null/);});
```

Listing 6: Node-dependent test

Listing 7 presents another flaky test that passes on Node.js version 18 but fails on versions 20 and 22. In Node.js version 18, the ES module system is more forgiving, especially in test environments that transpile JSX before running the code. However, Node.js versions 20 and 22 have stricter parsing rules and try to parse JSX as regular JavaScript, which causes syntax errors [28]. To resolve this issue, the JSX code must be transpiled appropriately (e.g., using Babel.js transpiler) before running the tests, or the test runner configuration must be updated to handle JSX correctly in newer Node.js versions.

```
it('should return a valid Provider Component', () => {
  const { Provider } = createContext();
  const contextValue = { value: 'test' };
  const children = [<div>child1</div>, <div>child2</div>];
  const providerComponent = <Provider {...contextValue}>{children}</Provider>; //expect(providerComponent).to.have.property('tag', 'Provider');
  expect(providerComponent.props.value).to.equal(contextValue.value);
  expect(providerComponent.props.children).to.equal(children);});
```

Listing 7: Node-dependent test using JSX

Of the projects we analyzed, we found that three of those projects declare a specific Node.js version or version range in their `package.json` file (via the `engines` field). They were different from the versions that we considered for the running environment and caused failures. We did not consider these failures as due to Node.js dependency issues. This choice ensures that we do not report expected incompatibility errors as instances of environmental flakiness.

3) Browser dependency: Some failures typically occur in browser-based or end-to-end tests, where inconsistencies between browser engines (e.g., Chromium vs. WebKit) or browser versions lead to flakiness. Errors can also be specific to certain combinations of browsers and OS (i.e., tests fail on specific browser setups within a specific OS). In total, we found 15 projects with browser-related flakiness, where eight of them are flaky as projects, while the remaining nine have Environmental flaky tests. The majority of browser-related flakiness–observed in eight projects–occurs on macOS, affecting tests executed in Chrome, Firefox, or Safari. These failures are typically caused by differences in browser engine behaviour, missing binaries, or variations in how browser APIs behave across platforms.

Listing 8 shows an example of a configuration file from the `axios` project where part of the test suite is configured to run using Firefox. On macOS, these tests fail with the error *"No binary for FirefoxHeadless browser on your platform"*, as the required Firefox binary is not available in the test environment. The error can be fixed by installing Firefox when running tests on macOS.

```
"test:karma": "node bin/ssl_hotfix.js cross-env LISTEN_ADDR=:: karma start karma.conf.cjs --single-run",
"test:karma:firefox": "node bin/ssl_hotfix.js cross-env LISTEN_ADDR=:: Browsers=Firefox karma start karma.conf.cjs --single-run",
"test:karma:server": "node bin/ssl_hotfix.js cross-env karma start karma.conf.cjs",
"test:build:version": "node ./bin/check-build-version.js",
```

Listing 8: package.json using Firefox browser

We can see a similar browser-related flakiness appears in the project `jQuery`, where a test fails exclusively on macOS (see Listing 9). The test, which checks the native abort behaviour of `jQuery.ajax()`, fails with the message "unexpected success". This message indicates that the test passed when a failure was expected, suggesting a platform-specific difference in how `XMLHttpRequest` abort events are handled in Chrome's headless mode on macOS. Such discrepancies can lead to unreliable outcomes in browser-based test suites across different environments.

```
ajaxTest( "jQuery.ajax() - native abort", 2, function( assert ) {
  return {
    url: url( "mock.php?action=wait&wait=1" ),
    xhr: function() {
      var xhr = new window.XMLHttpRequest();
      setTimeout( function() {
        xhr.abort();
      }, 100 );
      return xhr;
    },
    error: function( xhr, msg ) {
      assert.strictEqual( msg, "error", "Native abort triggers error callback" );
    },
    complete: function() {
      assert.ok( true, "complete" );}};} );
```

Listing 9: Native abort behaviour

4) Combination of Node.js and operating system configurations: Flakiness in this category can be attributed to the interaction of multiple environmental factors. For instance, a test may pass under most conditions but fail only on a specific OS with a particular version of Node.js or a browser. Such issues are more complex to diagnose as they involve multiple combinations of environmental factors, making them difficult to reproduce. Because of the number of cases and the importance of browsers in JavaScript, we consider the combination of browser and OS as part of the issue with browser, as discussed previously in Section IV-B3.

In this category, we identify a total of 16 projects with flakiness caused by the combination of OS and Node.js version. Of these projects, eight are Environmental flaky projects, while eight are Environmental flaky tests. In addition, 13 of these projects fail on Windows, eight on macOS, and five failed from combinations involving Ubuntu and Node.js version.

An example is shown in the *json-server* project (Listing 10), where the tests fail on Windows when run with Node.js versions 18 and 20. This failure transpired because, on Windows, the default command prompt (`cmd.exe`) does not perform glob pattern expansion. As a result, commands like `node --test src/*.test.ts` pass the pattern `*.test.ts` as a literal string to Node.js, causing errors as the test runner cannot resolve the intended files. Since version 22, an improved glob handling was introduced in the built-in test runner for Windows, which can mitigate this issue [29].

```
"scripts": {
    "dev": "tsx watch src/bin.ts ↩
        ↪ fixtures/db.json",
    "build": "rm -rf lib && tsc",
    "test": "node --import tsx/esm --test ↩
        ↪ src/*.test.ts",
    "lint": "eslint src",
    "prepare": "husky",
    "prepublishOnly": "npm run build"},
```

Listing 10: package.json file using command that not work on Windows with Node.js versions 18 and 20

V. Mitigation Strategy (RQ3)

To address flaky tests due to environmental factors, we propose a lightweight mitigation approach that allows running specific tests or test files under certain environmental configurations, enabling CI builds to succeed without rerunning entire test suites due to flaky failures, thereby reducing developer inspection. We designed this approach to be intentionally lightweight so that it does not require rerunning the tests (which is expensive) and also requires minimal configuration.

A previous study noted that when OS-specific conditions cause flakiness, developers often attempt to skip such OS-related flaky tests by embedding conditional logic to detect the current OS type and version within the test code, thereby allowing the tests to run only on specific OS [14]. A common industrial practice to deal with flaky tests is to *quarantine* them, i.e., allowing the CI to bypass the test while still reporting it as flaky [10], [30]. Several test monitoring tools also support quarantining flaky tests by identifying them from previous runs [31], [32], [33]. JUnit (for Java tests) provides a similar conditional test execution [34], which utilizes annotations to specify environmental conditions under which tests should (or should not) run. *@EnabledOnOs* and *@DisabledOnOs* are JUnit annotations that, respectively, enable and disable tests on specific OS. A GitHub code search shows more than 7,000 uses of *@EnabledOnOs* [35] and more than 6,000 uses of *@DisabledOnOs* [36]. A similar approach has also been adopted in a previous tool, *saflate*, which sanitizes flaky tests due to network dependencies (i.e., tests that fail due to network availability exceptions in JUnit) using custom annotations added to test files or tests [37].

Our approach employs a similar strategy for JavaScript tests, aiming to develop an expandable, cross-framework solution that works across popular JavaScript testing frameworks. It introduces comparable functionality through static code transformation, enabling developers to skip tests based on simple annotations in their test files. It also generates detailed reports explaining why tests were skipped and under what conditions. These reports describe the affected tests and environments, providing developers with a clear starting point for diagnosing and ultimately fixing the underlying issue.

A. Approach

The primary goal of our lightweight approach to address environment-specific flaky tests is to allow developers to manage them using annotations. Developers annotate their tests with docblock [38] tags, specifying the conditions under which a test should be skipped or enabled. This approach eliminates the need for ad-hoc conditional logic within test code, promoting cleaner and more maintainable test suites. All intentionally skipped tests are then reported in a generated test report, which shows which tests are skipped and the identified possible reason for the flakiness.

The approach is designed to support multiple JavaScript testing frameworks, without requiring modifications to existing test logic. The core components of the proposed approach are:

- **Annotation Parser:** Extracts metadata from test file docblocks, identifying *skip* or *enable* conditions.
- **Environment Detector:** Detects, at runtime, the current operating system, Node.js version, and the active browser.
- **Condition Evaluator:** Compares the extracted annotations with the detected environment to decide whether a test should be skipped.
- **Test Skipping Module:** Marks tests as skipped and logs skip reasons for review, before test execution.

The high-level workflow of the tool is as follows: when a test file is loaded, it parses all docblock annotations, evaluates them against the runtime environment, and automatically skips tests that do not meet the specified conditions. This design ensures consistent and automated enforcement of environment-specific constraints across different testing frameworks.

We have developed a set of annotation tags that can be used to enable/disable tests on specific environment configurations: OS types, Node.js versions, and browser types. Table IV

lists all the annotation tags we defined, along with their descriptions. Tag names and values are case-insensitive, and lists are comma-separated. Each test will run as the default unless a `skip` condition matches.

TABLE IV: Annotation tags supported by `js-env-sanitizer`

Type	Tag	Effect when condition holds
OS	`@enableOnOs` `@skipOnOs`	Run only (or skip) on the specified OS.
Node.js	`@enableOnNodeVersion` `@skipOnNodeVersion`	Run only (or skip) on the specified Node.js version(s).
	`@enableOnNodeRange` `@skipOnNodeRange`	Run only (or skip) when Node.js satisfies the given version range.
Browser	`@enableOnBrowser` `@skipOnBrowser`	Run only (or skip) when the active browser matches one of the specified values.

B. Implementation

We implemented our lightweight sanitization tool, `js-env-sanitizer`, as a Babel plugin[4], extending the test runner to intercept the execution pipeline before test execution. It processes the test suite by scanning the docblock annotations associated with each test. These annotations define constraints

The steps taken to run or skip tests are detailed in Algorithm 1. At first, `js-env-sanitizer` extracts the current runtime environment (i.e., OS, Node.js version, and the browser). Each test's annotations are then parsed and compared against this environment. If the conditions indicate that a test should not run (e.g., `@skipOnOS win32` when the OS is Windows), the test is marked as such in a test log saved in the project directory. Otherwise, the test is preserved for execution. This approach ensures that environment-specific tests are automatically skipped without manual intervention, improving test stability and reproducibility. The tool operates without modifying the original test code, facilitating easy integration into CI workflows.

The tool currently supports three testing frameworks: Jest, Mocha, and Vitest. However, the tool is extensible and can be readily expanded to support other testing frameworks.

C. Evaluation

We evaluate `js-env-sanitizer` using a subset of projects from our dataset (discussed in Section III-A) that (1) use one of the supported test frameworks (Jest, Mocha, or Vitest) and (2) contain at least one environmental flaky test. We start with 15 candidate projects. Among these projects, six use multiple testing frameworks or runners, and in those cases, we found that the environmental flaky tests occur in unsupported frameworks. For example, `swagger-ui` uses both Jest and Cypress tests. Upon investigation, we found that the two detected environmental flaky tests occurred in the Cypress test suite, which `js-env-sanitizer` does not currently support. We therefore exclude those six projects from the evaluation and report results on the remaining projects. The projects we included in our evaluation are three projects that use Jest, four that use Mocha, and two that use Vitest.

[4]https://babeljs.io/docs/plugins

Algorithm 1: Test sanitization using annotations

Input: Test suite
Output: Modified ineligible test blocks marked as skipped
Load test suite
Detect current OS, Node.js version, and browser (if applicable)
for each testing block in test suite **do** ▷ describe, it or test blocks
  if *it has supported annotation tags before it* **then**
    shouldSkip ← ! (environment matches all enable-conditions) ∧ (environment matches any skip-condition)
  end if
  if *shouldSkip* **then**
    Change the state of the test to skip
    Log skip reason with timestamp and matched condition
  end if
end for
Return test suite for execution

By declaring `js-env-sanitizer` under a project's `devDependencies`, the package auto-wires itself during installation and activates at test execution time without any manual setup. It detects the host test runner (Jest, Mocha, or Vitest) and attaches the appropriate lightweight integration (e.g., *babel-jest* transform, a minimal *@babel/core* require-hook for Mocha, or a vite Babel plugin for Vitest).

For each project, we add `js-env-sanitizer` as a dependency into the package and apply the appropriate annotation based on the experiments described in Section IV. We then push these changes to the forked projects and run the project's CI workflow, which executes the test suite across nine environment configurations, each repeated ten times.

In two projects, *insomnia* and *mongoose*, we were unable to evaluate this sanitization approach. *insomnia* features multiple subprojects and workspaces that share a common `Vitest` configuration. This made the instrumentation more challenging, as the test configuration file is not localized to a single package but instead inherited across different workspaces. Consequently, our setup script could not fully integrate with the heterogeneous project setup, and some tests executed before the sanitizer hooks could not be properly run. Similarly, *mongoose* uses`Mocha`, and most of its tests require a live *MongoDB* instance for execution. *mongoose* already includes its own mechanism to skip specific database-dependent tests when a required environment variable (`MONGOOSE_SHARD_TEST_URI`) is not set (an interestingly similar sanitization approach to ours that targets database configurations). However, once we added `js-env-sanitizer`, this built-in skipping mechanism did not take effect as expected, and the connection attempts were still triggered. In practice, the tests that should have been skipped by *mongoose*'s own logic were executed anyway, leading to failures. Therefore, we excluded the two projects from our

evaluation and reported the results for the other seven projects.

Table V summarizes the evaluation results. Shaded cells indicate the number of test failures due to environmental dependency, followed by the number of skipped tests after applying our sanitization tool. The detailed evaluation results are available in our replication package [39]. As shown in Table V, the results show that `js-env-sanitizer` can successfully sanitize and skip all tests that have failed due to environmental (OS, Node.js, or browser) dependencies. In other words, the initially failing tests have all been successfully quarantined and reported as skipped, allowing the CI build to continue without the known test failure.

To evaluate the tool's accuracy, we checked for any false positives (FPs), i.e., tests incorrectly skipped, and false negatives (FNs), i.e., tests that should have been skipped but were not. Based on our manual verification, we did not observe any FPs or FNs, indicating that the tool has a high accuracy. The tool is also shown to have a low performance overhead (see the runtime data in Table V; runtime values are averages across 90 runs). For most projects, we observe no significant difference in average runtime before and after sanitization. A notable exception is *fabric.js*, where the average runtime drops significantly as skipping the tests allowed the CI to continue, reducing the overall runtime.

Listings 11 demonstrate the use of annotation tags to skip environmental flaky tests, corresponding to the examples in Listings 7 and 4 in Section IV, respectively. The annotation tag "`@skipOnNodeVersion 20,22`" restricts execution of the test to Node.js version 18, and automatically skips the test on versions 20 and 22, where it is known to fail. Similarly, the annotation tag "`@skipOnOS win32`" ensures that the test is skipped on Windows, preventing OS-specific failures (due to the use of Unix-like paths) in Windows.

```
/**
  * @skipOnNodeVersion 20,22
  */
it('should return a valid Provider Component',

/**
  * @skipOnOS win32
  */
it('should output the correct snippet ids',
```

Listing 11: Annotation tags to skip tests on Node and Windows

VI. Discussion

Our systematic evaluation of environmental factors reveals that environmental flakiness is common in JavaScript projects and can manifest in various ways. For example, the whole project may fail due to environment-specific settings (at the package level), or only a few tests break under specific conditions. We have identified tests that can run under specific steps, such as with a particular OS, Node.js version, or browser (or a combination of these factors). Based on our results, many OS-dependent issues were specific to Windows, with path-related issues being the most common type. Browser-related problems mainly appeared when running the test in macOS.

Environmental flakiness can reveal deeper issues, such as features that are not equally supported across platforms or code that depends on OS-specific behavior. Developers often need to run tests in multiple environments, and when failures occur, they should reproduce the problem in the same environment where it originally happened. Our approach encourages cross-environment testing, allowing developers to control which tests can be run across different environmental setups without the need to change the test logic.

The root causes we identified point to practical lessons. For OS-dependent issues, cross-platform practices such as using `cross-env` for environment variables or normalizing file paths make tests more reliable. For Node.js-dependent issues, it helps to clearly specify supported versions in `package.json` and avoid fragile assertions that depend on exact error messages. For browser-dependent failures, making sure the right browser binaries are installed in CI and writing tests that work across different engines reduces surprises.

VII. Threats to Validity

The study uses open-source JavaScript projects available on GitHub. The observations regarding environmental flakiness are limited to the selected projects and may not be fully generalizable to all JavaScript projects. To mitigate the threat of generalizability, we selected popular (ranked by stars) repositories maintained by large communities and spanning diverse application domains, including servers, front-end and back-end frameworks, and web applications. The selected projects also use a variety of testing frameworks.

Part of the project selection and the analysis of the cause of flakiness was performed manually, which may introduce construct validity threats due to potential oversight or bias. The manual analysis consisted of two stages: (1) installing and investigating the tests in each project to observe and classify the impact on environmental flakiness, and (2) installing `js-env-sanitizer` and manually adding annotation tags to skip the environmental dependent tests under the corresponding environments. To mitigate bias, two authors independently examined the source code and error messages to reduce false positives in classification. In cases of disagreement, a third author independently reviewed the instance, and all three authors discussed the causes until consensus was reached.

VIII. Related Work

A large number of studies investigated various aspects of flaky tests, including root causes and methods for identifying, detecting, predicting, or mitigating them [11], [40], [10]. Previous studies have focused mainly on specific causes of test flakiness, such as test-order dependency [41], [42], [43], concurrency [44], [45], [46], unordered collections [19], code instrumentation [47], or network flakiness [37].

A. Empirical Studies on Test Flakiness

Luo et al. [12] reported the first empirical study on flaky tests in Java projects. The study classified the root causes of flaky tests into ten main categories, noting that one of

TABLE V: Results of sanitizing environmental flaky tests using `js-env-sanitizer`

Framework	Project	#tests	Default tests run					Sanitized Tests			
			#failure	#skipped	#passed	Avg runtime (s)	Cause	#failure	#passed	#skipped	Avg runtime (s)
Jest	iptv	8	8	0	7	31	OS & Node	0	8	8	77
	fabric.js	162	99	2	61	1020	OS & Node	0	61	101	65
	pug	650	2	0	648	59	Node range	0	648	2	85
Mocha	rollup	5213	1	0	5212	718	OS & Node	0	5212	1	721
	sails	1550	1	13	1536	237	Node	0	1536	14	238
	preact	1217	1	13	1203	68	Node	0	1203	14	147
Vitest	30-seconds-of-code	148	17	0	131	40	OS	0	131	17	63

these causes is environment dependency. Eck et al. [40] also cited *platform dependency*, which describes flaky behavior that occurs only on specific platforms (OS or virtual environment).

Hashemi et al. [14] examined root causes and how developers attempt to address flaky tests in JavaScript. One of the main causes reported is OS-dependency, in which a test passes on one OS but fails on another. The study reported that developers usually choose to skip, ignore, or disable the problematic tests by adding conditions to the tests that check the underlying OS before executing them. Similarly, Job et al. [21] studied OS-specific tests in Python, and found that OS-specific tests are common (56% of the analyzed projects), with Windows-specific tests being the most common type.

B. Flakiness Caused by Environmental Factors

Environmental factors, such as OS, external libraries, or compilation environments, can significantly influence test behavior, often leading to non-deterministic outcomes [48], [49]. Such variability is a common source of flakiness, as tests that pass in one environment may fail in another due to subtle differences in system behavior or resource availability. Developers frequently introduce OS-specific tests to account for these variations [14], performing operations such as calling platform-dependent APIs, mocking OS-level objects, or controlling execution flow. Job et al. [21] show that OS-specific tests are often employed to work around missing external resources, unsupported standard libraries, and flaky tests.

A study on SAP HANA's CI pipeline [50] emphasizes the significant impact of the testing environment on test flakiness. It has been reported that environmental factors, including the execution infrastructure and test distribution, play a significant role in producing flaky behavior. While higher test execution times, possibly due to limited infrastructure speed, correlated with increased flakiness, other factors such as system load, memory, and CPU usage did not show a clear relationship.

FlakyLoc [51] aims to identify the root causes of flakiness in web applications by executing known flaky tests across different environmental configurations (e.g., memory size, browsers, and screen resolution), which vary across executions.

C. Flaky Tests in JavaScript

An empirical study of flaky tests in JavaScript projects identified concurrency, async wait, and OS as the top three causes of test flakiness [14]. Regarding how flaky tests are handled, the majority of such tests (> 80%) are fixed by developers to eliminate flaky behavior. Costa et al. [15] investigated flaky tests across five languages, including JavaScript, and found that *async wait*, *concurrency*, and *platform dependency* were the top causes of flakiness in JavaScript.

Heshemi et al. [42] investigated order-dependent tests in JavaScript by systematically reordering and rerunning tests and test suites. The study found that order-dependent tests are caused by either shared file or shared mocking states between tests. Yost [52] employed a combination of stress testing and test suite reordering to detect flaky tests in JavaScript applications. The proposed method involved subjecting the system to stress and altering the execution order of test cases to expose issues that may remain undetected under standard conditions. FlakyFix [53] utilized large language models to classify the types of fixes applied to flaky tests. The proposed approach examines test code to uncover patterns that contribute to flakiness, enabling the prediction of fix categories.

There are only a few dedicated flaky test tools for JavaScript [54], [10]. Of those tools, NodeRacer [55] focuses on detecting flakiness due to concurrent, event races in Node.js applications. JS-TOD [56] focuses on detecting order-dependent flaky tests in JavaScript by systematically reordering and rerunning tests.

IX. Conclusion

In the study, we present an in-depth systematic evaluation of environmental factors in JavaScript projects. We analyzed possible flaky behavior caused by variation in the test environment, considering three environmental factors: OS type, Node.js version, and browser type. Our evaluation shows that OS-dependency is the most common type of environmental flakiness and happens across testing frameworks (28). This includes issues related to path formatting (between Windows and Unix-like systems) and the use of OS-specific options. There are also other issues related to Node.js versions (i.e., the use of unsupported features in older versions) or the use of a specific browser in a specific OS.

To address these issues, we developed `js-env-sanitizer`, a lightweight annotation-based sanitization tool that supports Jest, Mocha, and Vitest testing frameworks. The tool allows developers to define tests that run only on a

specific environment, enabling tests to skip if the environment differs from the defined configuration. This approach allows CI builds to continue and succeed without rerunning entire test suites, while also reporting and documenting the intentionally skipped tests for future fixes.

REFERENCES

[1] N. Alshahwan, M. Harman, and A. Marginean, "Software testing research challenges: An industrial perspective," in *2023 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2023.

[2] M. Machalica, W. Chmiel, S. Swierc, and R. Sakevych, "Probabilistic flakiness: How do you test your tests? - engineering at Meta," https://engineering.fb.com/2020/12/10/developer-tools/probabilistic-flakiness/, 2020, (Accessed on 12/03/2024).

[3] M. Fowler, "Eradicating non-determinism in tests," https://martinfowler.com/articles/nonDeterminism.html, (Accessed on 10/02/2024).

[4] J. Palmer, "Test flakiness – methods for identifying and dealing with flaky tests - spotify," 2019. [Online]. Available: https://engineering.atspotify.com/2019/11/18/test-flakiness-methods-for-identifying-and-dealing-with-flaky-tests/

[5] M. Machalica, A. Samylkin, M. Porth, and S. Chandra, "Predictive test selection," in *Proceedings of the International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019.

[6] W. Lam, A. Shi, R. Oei, S. Zhang, M. D. Ernst, and T. Xie, "Dependent-test-aware regression testing techniques," in *Proceedings of the ACM International Symposium on Software Testing and Analysis (ISSTA)*, 2020.

[7] C. Ziftci and D. Cavalcanti, "De-flake your tests: Automatically locating root causes of flaky tests in code at google," in *2020 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2020.

[8] S. Thummalapenta, "Improving developer productivity via flaky test management engineeringmicrosoft," 2022, (Accessed on 01/09/2025). [Online]. Available: https://devblogs.microsoft.com/engineering-at-microsoft/improving-developer-productivity-via-flaky-test-management/

[9] J. Raine, "Reducing flaky builds by 18x - the GitHub blog," https://github.blog/engineering/reducing-flaky-builds-by-18x/, 2021, (Accessed on 31/03/2024).

[10] A. Tahir, S. Rasheed, J. Dietrich, N. Hashemi, and L. Zhang, "Test flakiness' causes, detection, impact and responses: A multivocal review," *Journal of Systems and Software*, vol. 206, p. 111837, 2023.

[11] S. Thorve, C. Sreshtha, and N. Meng, "An empirical study of flaky tests in android apps," in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2018.

[12] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, "An empirical analysis of flaky tests," in *Proceedings of the ACM International Symposium on Foundations of Software Engineering (FSE)*, 2014.

[13] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, "An empirical study of flaky tests in python," in *Proceedings of the IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2021.

[14] N. Hashemi, A. Tahir, and S. Rasheed, "An empirical study of flaky tests in javascript," in *Proceedings of the IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 2022.

[15] K. Costa, R. Ferreira, G. Pinto, M. d'Amorim, and B. Miranda, "Test flakiness across programming languages," *IEEE Transactions on Software Engineering*, 2022.

[16] "Jest," (Accessed on 01/09/2025). [Online]. Available: https://jestjs.io/

[17] "Mocha," (Accessed on 01/09/2025). [Online]. Available: https://mochajs.org/

[18] "Vitest," (Accessed on 01/09/2025). [Online]. Available: https://vitest.dev/

[19] A. Shi, A. Gyori, O. Legunsen, and D. Marinov, "Detecting assumptions on deterministic implementations of non-deterministic specifications," in *2016 IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2016.

[20] A. Adekotujo, A. Odumabo, A. Adedokun, and O. Aiyeniko, "A comparative study of operating systems: Case of windows, unix, linux, mac, android and ios," *International Journal of Computer Applications*, vol. 176, no. 39, pp. 16–23, 2020.

[21] R. Job and A. Hora, "How and why developers implement os-specific tests," *Empirical Software Engineering*, vol. 30, no. 1, pp. 1–33, 2025.

[22] Stack Overflow, "Stack overflow developer survey 2024: Web frameworks and technologies," https://survey.stackoverflow.co/2024/technology#1-web-frameworks-and-technologies, 2024, (Accessed on 01/09/2025).

[23] GitHub, "Github actions now supports ci/cd," https://github.blog/news-insights/product-news/github-actions-now-supports-ci-cd/, 2019, (Accessed on 01/09/2025).

[24] "REST API endpoints for search - GitHub docs," https://docs.github.com/en/rest/search/search, (Accessed on 10/01/2025).

[25] S. Brisson, E. Noei, and K. Lyons, "We are family: analyzing communication in GitHub software repositories and their forks," in *Proceedings of the IEEE 27th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 2020.

[26] "cross-env: Run scripts that set and use environment variables across platforms," https://www.npmjs.com/package/cross-env, (Accessed on 01/09/2025).

[27] M. Dawson. (2017) Node.js errors changes you need to know about. (Accessed on 09/06/2025). [Online]. Available: https://medium.com/the-node-js-collection/node-js-errors-changes-you-need-to-know-about-dc8c82417f65

[28] "Node.js v20.0.0 release: Custom esm loader hooks run on dedicated thread," https://nodejs.org/en/blog/release/v20.0.0/#custom-esm-loader-hooks-run-on-dedicated-thread, 2023, (Accessed on 01/09/2025).

[29] "Discussion on node.js issue #50658: Comment on esm loader and threading behavior," https://github.com/nodejs/node/issues/50658#issuecomment-1806581766, (Accessed on 01/09/2025).

[30] S. Habchi, G. Haben, M. Papadakis, M. Cordy, and Y. L. Traon, "A qualitative study on the sources, impacts, and mitigation strategies of flaky tests," *arXiv preprint arXiv:2112.04919*, 2021.

[31] B. Chen and B. Lee, "Flaky tests: their hidden costs and how to address flaky behavior," Online; Datadog Engineering blog, 2024, accessed: 2025-12-16. [Online]. Available: https://www.datadoghq.com/blog/datadog-flaky-tests/#quarantine-tests-that-frequently-flake

[32] N. Malik, "Taming test flakiness: How we built a scalable tool to detect and manage flaky tests," Online; Atlassian, 2025, accessed: 2025-12-16. [Online]. Available: https://www.atlassian.com/blog/atlassian-engineering/taming-test-flakiness-how-we-built-a-scalable-tool-to-detect-and-manage-flaky-tests

[33] "Trunk," 2025, accessed: 2025-12-16. [Online]. Available: https://docs.trunk.io/flaky-tests/quarantining

[34] "Junit 5 user guide: Writing tests âĂŞ conditional execution," (Accessed on 01/09/2025). [Online]. Available: \url{https://docs.junit.org/current/user-guide/#writing-tests-conditional-execution}

[35] "Github code search results for @enabledonos in java," https://github.com/search?q=@EnabledOnOs+language%3AJava&type=code, (Accessed on 10/09/2025).

[36] "Github code search results for @disabledonos in java," https://github.com/search?q=@DisabledOnOs+language%3AJava&type=code, (Accessed on 01/09/2025).

[37] J. Dietrich, S. Rasheed, and A. Tahir, "Flaky test sanitisation via on-the-fly assumption inference for tests with network dependencies," in *Proceedings of the IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*, 2022.

[38] "Javascript docblock coding standards," https://developer.adobe.com/commerce/php/coding-standards/js-docblock/, (Accessed on 01/09/2025).

[39] Anonymous, "Replication package for the paper: A systematic evaluation of environmental flakiness in javascript tests," 2025, https://anonymous.4open.science/r/env-flakiness-js-BE44.

[40] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, "Understanding flaky tests: The developer's perspective," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2019.

[41] A. Gambi, J. Bell, and A. Zeller, "Practical test dependency detection," in *Proceedings of IEEE International Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 2018.

[42] N. Hashemi, A. Tahir, S. Rasheed, A. Shi, and R. Blagojevic, "Detecting and evaluating order-dependent flaky tests in javascript," *arXiv preprint arXiv:2501.12680*, 2025.

[43] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, "idflakies: A framework for detecting and partially classifying flaky tests," in *Proceedings of the 2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, 2019.

[44] S. Rahman and A. Shi, "FlakeSync: Automatically repairing async flaky tests," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024.

[45] S. Rahman, A. Massey, W. Lam, A. Shi, and J. Bell, "Automatically reproducing timing-dependent flaky-test failures," in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, 2024.

[46] Z. Dong, A. Tiwari, X. L. Yu, and A. Roychoudhury, "Flaky test detection in android via event order exploration," in *Proceedings of the ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2021.

[47] S. Rasheed, J. Dietrich, and A. Tahir, "On the effect of instrumentation on test flakiness," in *Proceedings of IEEE/ACM International Conference on Automation of Software Test (AST)*, 2023.

[48] J. Micco, "The state of continuous integration testing at google," https://testing.googleblog.com/2017/04/where-do-our-flaky-tests-come-from.html, 2017.

[49] V. Terragni, P. Salza, and F. Ferrucci, "A container-based infrastructure for fuzzy-driven root causing of flaky tests," in *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: New Ideas and Emerging Results*, 2020.

[50] A. Berndt, T. Bach, and S. Baltes, "Do test and environmental complexity increase flakiness? an empirical study of sap hana," in *Proceedings of the 18th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement*, 2024.

[51] J. Morán Barbón, C. Augusto Alonso, A. Bertolino, C. A. Riva Álvarez, J. F. García Tuya *et al.*, "Debugging flaky tests on web applications," in *Proceedings of the 15th International Conference on Web Information Systems and Technologies-Volume 1: APMDWE*, 2019.

[52] G. A. Yost, "Finding flaky tests in javascript applications using stress and test suite reordering," Master's thesis, The University of Texas at Austin, 2023.

[53] S. Fatima, H. Hemmati, and L. Briand, "FlakyFix: Using large language models for predicting flaky test fix categories and test code repair," *arXiv preprint arXiv:2307.00012*, 2024.

[54] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, "A survey of flaky tests," *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 31, no. 1, 2021.

[55] A. T. Endo and A. Møller, "NodeRacer: Event race detection for node.js applications," in *Proceedings of the IEEE International Conference on Software Testing, Validation and Verification (ICST)*, 2020.

[56] N. Hashemi, A. Tahir, S. Rasheed, A. Shi, and R. Blagojevic, "Jstod: Detecting order-dependent flaky tests in jest," *arXiv preprint arXiv:2509.00466*, 2025.